Defeating Context Rot: Recursive Retrieval for Long-Document Question Answering

March 22, 2026

Abstract

Large language models can fail when important facts are buried in long documents, a problem often called *context rot*. This project evaluates a more paper-faithful but still prompt-based Recursive Language Model (RLM) controller that searches an external document store, reads small spans, and can recursively query subcontexts under a fixed budget. We compare it with Full Context, RAG, and Map-Reduce on Needle-in-Haystack, synthetic multi-hop QA, LongBench subsets, and MuSiQue. Across 2,003 retained evaluations at $10.0257 tracked cost, results are mixed: RLM leads only on the retained needle suite, RAG is stronger on synthetic multi-hop and MuSiQue, and Full Context leads the evaluated LongBench slice. A later harder 500k needle diagnostic exposed clearer context-rot behavior, but it still favored RAG over RLM. Overall, this reproduction is closer to the RLM paper in method than in outcome.

1 Introduction

Long-document QA is difficult even when a model has a large context window. Accuracy can still drop when the answer is buried deep in the text or when the task requires linking distant facts. Recursive Language Models (RLMs) address this by treating the document as an external environment that the model can inspect step by step instead of reading everything in one pass [1,2]. This project implements that idea as faithfully as practical in an API-based system and compares it with standard long-context baselines, especially one-shot retrieval-augmented generation [3].

2 Literature Review

Prior work suggests that long-context failures are not solved by larger windows alone. Liu et al. show that models are often weakest when relevant evidence appears in the middle of a long prompt, motivating the context-rot problem studied here [1]. RAG addresses this by moving evidence outside the prompt and retrieving once before answering, but that single-pass design can still struggle on multi-hop or compositional questions [3]. Recursive alternatives try to solve this by breaking long reasoning into smaller steps: Wu et al. show that hierarchical summarization can manage long inputs, and Zhang et al. formalize this idea with Recursive Language Models that inspect an external environment recursively [2, 6]. LongBench and MuSiQue then provide useful evaluation settings for broad long-context understanding and explicit multi-hop reasoning [4, 5]. The gap for this project is practical: there is strong motivation for recursive control, but fewer clear reproductions of prompt-based RLM-style systems under fixed API budgets.

3 Method and Setup

The RLM controller operates through a small tool interface:

- `search(query, top_k)` returns ranked candidate chunks.
- `read_chunks(...)` returns the text of selected chunks.
- `read_span(start, end)` returns a targeted character span.
- `chunk_span(chunk_id)` exposes a chunk's location in the source document.
- `llm_query(...)` recursively asks a subquestion over a smaller derived context.
- `finish(answer, confidence, reasoning)` terminates the search with a short answer.

The final controller is a bounded REPL-style policy rather than the earlier fixed `search` → `read` → `reason` → `merge` scaffold. Documents stay outside the prompt, the model reads only small spans, and recursive sub-calls are allowed on narrower contexts. To keep runs comparable, the controller enforces:

- maximum recursion depth of 4,
- maximum 8 REPL steps per query,
- bounded retrieval and read limits,
- full token, call, and dollar tracking.

This is closer to the RLM paper's external-environment paradigm than the initial scaffold, but it is still not a perfect reproduction: it uses Gemini API models and prompting rather than a natively trained recursive model.

From an engineering perspective, the implementation is end to end rather than a notebook-only prototype. The system includes cached retrieval artifacts, per-method incremental result saving, trace logs for each sample, explicit budget accounting, retry/backoff for transient API failures, and fail-fast checks for missing API credentials. These guard rails improve recoverability, but they do not guarantee completion under sustained provider-side throttling or availability problems; that is why the Pro MuSiQue follow-up still ended with an incomplete retained slice. The repository currently passes 53 automated tests across retrieval, benchmarking, controller behavior, and runner utilities.

All canonical runs use the following model stack:

- Fast model: `gemini-2.0-flash`
- Pro model: `gemini-2.5-pro`
- Embedding model: `gemini-embedding-001`

Four methods are compared under the same benchmark framework:

- **Full Context**: stuff as much source text as possible into one prompt.
- **RAG**: retrieve top-k chunks once, then answer once.
- **Map-Reduce**: summarize retrieved chunks, then answer from summaries.

- **RLM**: recursively inspect, read, and query smaller contexts.

The evaluation suite contains both synthetic and real-world tasks:

- **Needle-in-Haystack**: 210 examples per method over six haystack lengths and seven answer positions.
- **Synthetic Multi-Hop**: 120 examples per method across two hop counts and four document lengths.
- **LongBench** [4]: 100 examples per method from `qasper` and `narrativeqa`.
- **MuSiQue** [5]: 90 examples per method across 2-hop, 3-hop, and 4-hop settings.
- **MuSiQue Pro subset**: a partial higher-cost follow-up using the Pro model.

The retained canonical outputs cover 2,003 of 2,020 evaluations at $10.0257 tracked cost. Coverage is nearly complete on the main benchmarks; the main incomplete area is the MuSiQue Pro follow-up, where API failures left an asymmetric retained slice.

4 Results

4.1 Canonical Benchmarks

Table 1 summarizes the mean F1 scores for each benchmark. The results do not show a uniform RLM advantage.

Table 1: Canonical head-to-head mean F1 by benchmark and method. Best result in each row is bolded. The Pro follow-up is excluded here because the retained RAG and RLM slices are not directly matched.

Benchmark	Full Context	RAG	Map-Reduce	RLM
Needle-in-Haystack	0.900	0.900	0.896	**0.939**
Synthetic Multi-Hop	0.450	**0.795**	0.317	0.666
LongBench	**0.543**	0.513	0.526	0.484
MuSiQue	–	**0.445**	–	0.352

Needle-in-Haystack is the only canonical benchmark where RLM clearly leads. It improves mean F1 from 0.900 for both Full Context and RAG to 0.939. However, Full Context also stays unusually strong and nearly flat across positions, which suggests that the retained needle suite is too weak to expose strong context rot.

On the other canonical benchmarks, RLM does not lead overall. On synthetic multi-hop, the added 500k rows narrow but do not reverse the gap: RAG reaches 0.795 while RLM reaches 0.666. On LongBench, Full Context leads with 0.543. On standard MuSiQue, RAG leads 0.445 to 0.352.

The new 500k multi-hop conditions are now included in the canonical figure. At 2-hop / 500k, Full Context reaches only 0.067 and Map-Reduce remains at 0.000, while RAG holds 0.867 and RLM reaches 0.733. At 3-hop / 500k, RLM is strongest at 0.960, ahead of Full Context at 0.800, Map-Reduce at 0.733, and RAG at 0.533.

The zero-heavy 2-hop slice deserves explanation because it looks like a potential bug at first glance. Trace inspection suggests that it is mainly a benchmark-formulation failure mode rather than a missing-run artifact. The 2-hop question asks where a person's workplace was located, but the supporting facts separate employer identity from employer headquarters location. Full Context and especially Map-Reduce often stop at the intermediate organization name rather than the final location. For example, on a 2-hop / 10K sample, Map-Reduce returns "Vanguard Academy" when the gold answer is the headquarters location. This means the flat 0.00 Map-Reduce scores on the shorter 2-hop slices should be interpreted cautiously: they reflect a benchmark/method interaction, not an empty-output pipeline failure.

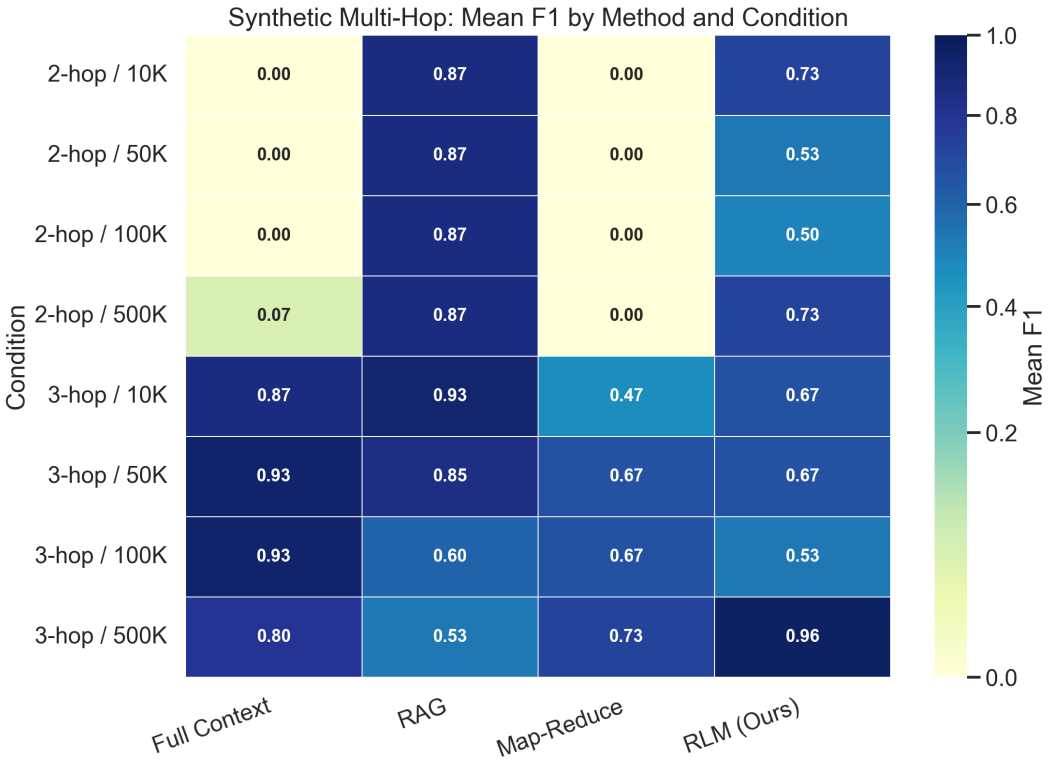

Figure 1: Synthetic multi-hop mean F1 heatmap by method and condition. The 500k rows are included, and the color scale is power-normalized so low-end differences remain visible.

LongBench is also more modest than the headline table alone suggests. The overall scores are pulled down mostly by the QASPER subset, where all methods fall into a narrow 0.34–0.38 range, while NarrativeQA is materially higher. Part of that gap is task difficulty, but part is evaluation mismatch: the project scores short answer-only outputs with token-level F1, which is harsher on paraphrastic scientific-paper answers than on short factual lookups.

The Pro MuSiQue follow-up also needs care. Its retained raw means are 0.489 for RAG and 0.644 for RLM, but those rows are not a clean head-to-head comparison. The retained RAG rows span 2-, 3-, and 4-hop settings, whereas the retained RLM rows cover only the 2-hop slice. On the 23 overlapping completed sample IDs, RAG edges out RLM, 0.670 to 0.644. Table 2 summarizes that audit.

Table 2: Audited view of the retained MuSiQue Pro follow-up.

Comparison Slice	RAG Mean F1	RLM Mean F1	Interpretation
Retained raw completed rows	0.489	0.644	Asymmetric slices
Matched overlap ($n = 23$)	0.670	0.644	RAG slightly leads

4.2 Stress Test and Failure Analysis

To stress the too-easy retained needle suite, a later post-canonical diagnostic reran the harder 500k-word challenging needle slice with dense near-miss distractors. That setting finally induced visible degradation in Full Context and produced a more believable stress regime, but it did not rescue RLM. On this slice, Full Context reached F1 0.722, RAG reached 0.778, Map-Reduce reached 0.577, and the pre-refresh RLM reached 0.695. A subsequent RLM-only fairness rerun that widened the bootstrap retrieval and added a final exact-answer verifier made things worse rather than better, dropping RLM to 0.463. Table 3 summarizes that diagnostic. These numbers are reported separately because they were run after the canonical benchmark table was finalized and the after-fix rerun only covered RLM.

Table 3: Post-canonical 500k challenging needle diagnostic on the same nine-sample slice.

Method / Variant	Mean F1
Full Context	0.722
RAG	0.778
Map-Reduce	0.577
RLM (pre-refresh)	0.695
RLM (after wider bootstrap + final verifier)	0.463

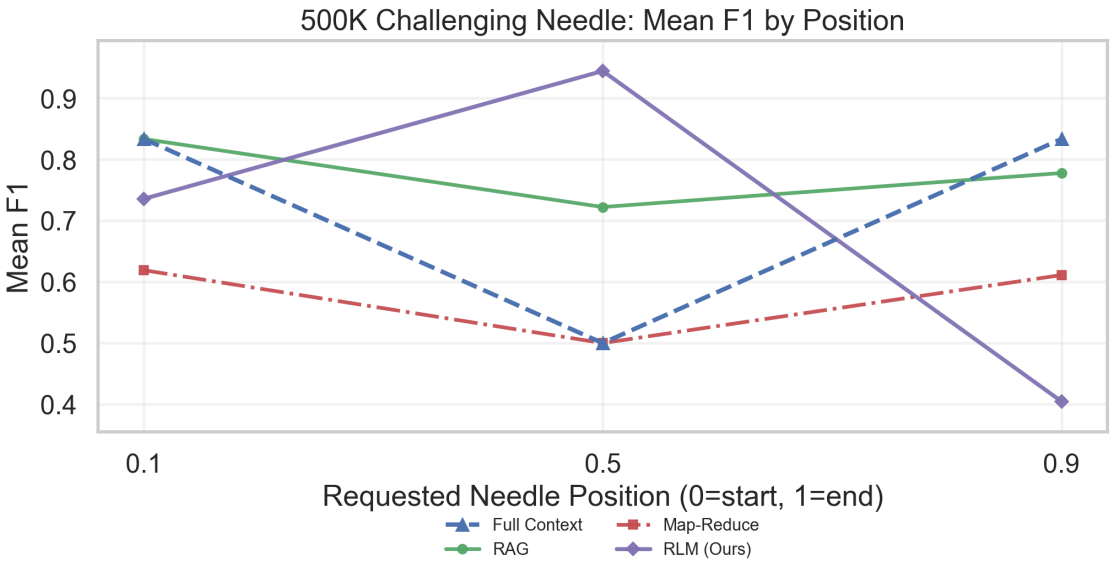

Figure 2: Post-canonical 500k challenging needle diagnostic by requested answer position. Unlike the retained canonical needle suite, this harder slice produces a clear mid-document dip for Full Context and a stronger separation between simple retrieval and recursive control.

Trace analysis explains why the added final verifier hurt rather than helped. In a representative 500k challenging sample, the controller reads a chunk containing the exact answer "162 degrees Celsius for 47 minutes," then recursively reframes the task as summarizing conflicting alternatives and finally returns "Conflicting information." The verifier does not recover the canonical fact; it reinforces the ambiguity framing. This is a useful negative result because it shows a concrete prompt-based RLM failure mode: once the recursive controller drifts into conflict summarization, extra verification can amplify that drift instead of correcting it.

4.3 Main Takeaways

The final results support a narrower claim than the RLM paper's strongest empirical statement:

- the retained needle suite under-stresses context rot,
- the later 500k challenging slice is a more credible stress test and favors RAG,
- recursion helps on some localization-heavy cases but does not win the main multi-hop averages here,
- the economics are poor for this implementation: RLM averages 49.6 seconds and 21.3 LLM calls per completed sample versus 2.7 seconds and 4.8 calls for RAG, while also trailing RAG in mean F1,
- the controller is sensitive to distractor-heavy evidence handoff,
- these numbers evaluate prompt-based recursive control, not a natively trained recursive model,
- the retained Pro raw averages are not clean evidence because the retained slices are not matched.

5 Cost and Efficiency

The retained canonical run covers 2,003 of 2,020 evaluations, with 17 non-ok rows. Its tracked cost is $10.0257 over 19,564 LLM calls, using 93,743,980 input tokens and 896,226 output tokens. Earlier in the project, the user reported $146.47 in billed spend before the later 500k multi-hop addendum, so the real billed total is higher than the tracked canonical total.

Per-benchmark cost is shown below:

- Needle-in-Haystack: $1.1770 across 4,030 calls
- Synthetic Multi-Hop: $3.2528 across 7,737 calls
- LongBench: $0.3507 across 2,954 calls
- MuSiQue: $0.4092 across 2,882 calls
- MuSiQue (Pro): $4.8361 across 1,961 calls

Table 4 shows method-level averages over completed samples.

Table 4: Method-level efficiency averages over completed rows.

Method	Mean F1	Mean Cost ($)	Mean Time (s)	Mean LLM Calls
Full Context	0.691	0.007	3.885	1.000
RAG	0.689	0.002	2.695	4.801
RLM	0.684	0.008	49.620	21.308
Map-Reduce	0.648	0.001	10.931	10.207

RLM remains the slowest method per sample, and it still does not deliver the best average F1. Its completed-sample mean F1 is 0.684, slightly behind both Full Context (0.691) and RAG (0.689).

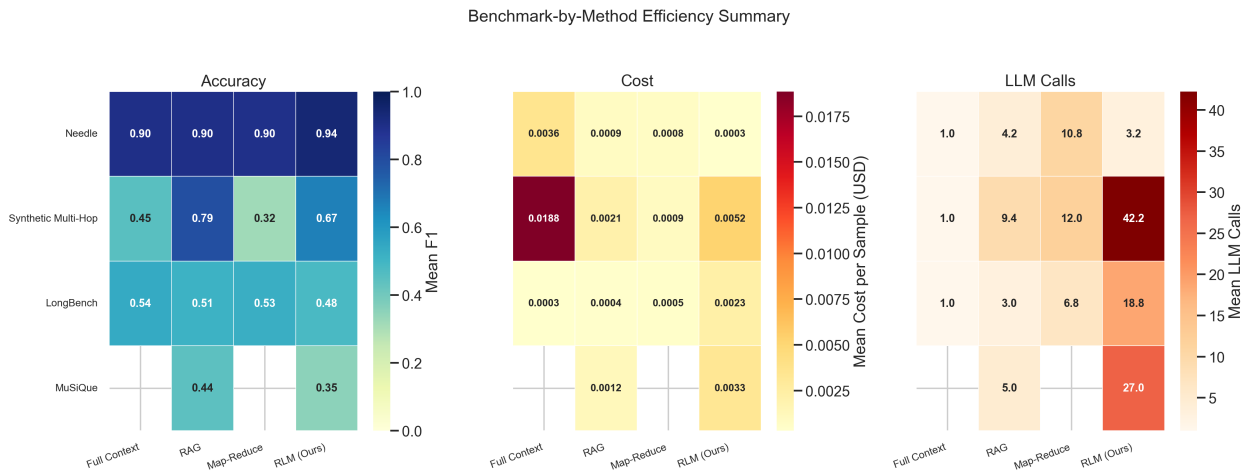

Figure 3: Benchmark-by-method efficiency summary. RLM provides the deepest reasoning path but at materially higher cost and call count.

6 Discussion

Methodologically, the project is much closer to the RLM paper than the initial scaffold. The controller treats the document as an external environment, uses targeted inspection instead of monolithic prompting, and recursively calls itself on smaller contexts [2]. Empirically, however, the gains are narrower than the paper's headline claim. The fairest reading is that this reproduction captures the paper's mechanism better than before, but not its aggregate outcome. More specifically, it evaluates *prompt-based RLM control* rather than a natively trained recursive model, so its negative results should be read as evidence about this implementation choice as much as about the paradigm itself.

The main reasons are straightforward:

- this project uses Gemini models rather than the paper's model stack,
- the controller is prompted into recursive behavior rather than post-trained for it,
- the retained needle suite was too easy and only later supplemented with a harder slice,
- the harder distractor setting exposed brittle evidence handoff in the current controller,

- the Pro MuSiQue follow-up is incomplete and not directly matched across methods.

The clearest next steps are to rerun on the harder needle setup, tighten evidence handoff in the recursive controller, and expand evaluation with a cleaner Pro or broader LongBench run.

7 Conclusion

This project tested whether prompt-based recursive retrieval could beat common long-context baselines under a fixed budget. The answer is mixed. RLM helps on the retained needle suite, but RAG is stronger on synthetic multi-hop and MuSiQue, and Full Context leads the evaluated LongBench slice. The later 500k challenging needle diagnostic is the more believable context-rot test, and it still favors RAG over RLM. The main practical lesson is not just that this RLM variant underperforms on average, but that it does so at substantially higher latency and call count. In this Gemini-based reproduction, recursive retrieval supports the paper's intuition more than its strongest empirical claim.

References

[1] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 2024.

[2] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive Language Models. arXiv:2512.24601, submitted December 31, 2025, revised January 28, 2026. https://arxiv.org/abs/2512.24601

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 2020.

[4] Yushi Bai, Xin Lv, Jiapeng Zhang, Hongyu Lin, and others. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv, 2023/2024.

[5] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

[6] Jeff Wu, Long Ouyang, Daniel M. Ziegler, Nisan Stiennon, Ryan Lowe, Jan Leike, and Paul Christiano. Recursively Summarizing Books with Human Feedback. arXiv, 2021.